Music Genre Classification

Kaggle Competition — Messy Mashup

Roll No: 24f1002643 | IIT Madras | Jan 2026

1. Abstract

This report presents the work done for the Jan 2026 Deep Learning and Generative AI Kaggle competition, which tasked participants with classifying audio clips into one of ten music genres. The dataset, named Messy Mashup, contained separated audio stems (vocals, drums, bass, other) per song, which posed an interesting challenge for both feature engineering and model design. Three distinct deep learning architectures were developed and evaluated — a fine-tuned Audio Spectrogram Transformer (AST), a custom residual CNN, and a hybrid CNN+Bidirectional GRU model. All training was done on GPU using PyTorch with mixed-precision, and synthetic data augmentation was applied to improve robustness. The primary metric used for evaluation was Macro F1 Score. The AST based model outperformed the other two, achieving the highest validation F1, while the CNN+GRU model showed surprisingly competitive results considering its far lower parameter count. Experiment tracking was done via Weights & Biases.

2. Introduction

2.1 Problem Statement

The competition required building a model that can identify the genre of an audio clip from the following ten classes: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. What made this slightly unusual compared to a standard classification task is that the training audio was not provided as raw mixed tracks — it was provided as separated stems. Each "song" had four component files: vocals.wav, drums.wav, bass.wav, and other.wav. The test set however consisted of mixed files, so any model had to generalize from stem-based training to mixed audio at inference time.

2.2 Project Objectives

The main objectives I set out to achieve were:

- Explore how pre-trained transformer models, specifically the Audio Spectrogram Transformer, can be fine-tuned effectively on a relatively small music genre dataset.
- Design a lightweight custom architecture (CNN-based) that can act as a strong baseline without requiring large compute.
- Understand how different augmentation strategies — SpecAugment, noise injection, synthetic mashup generation — affect model robustness.
- Compare the three approaches systematically on validation metrics.

2.3 Report Structure

Section 3 describes the dataset and preprocessing pipeline. Section 4 discusses the feature extraction strategy. Sections 5.1 through 5.3 detail each model architecture. Section 6 presents comparative analysis and results. Section 7 concludes with learnings and future directions.

3. Dataset & Preprocessing

3.1 Dataset Description

The competition dataset ("messy_mashup") contains audio recordings organized by genre, with each track split into four stems using a source separation tool (likely Demucs or a similar model). The 10 genres present are blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. In addition to the genre stems, the dataset also includes ESC-50 environmental sound clips which were used as background noise during augmentation. The test set consists of single-channel mixed audio files listed in a test.csv file.

3.2 Exploratory Data Analysis

Each genre folder contains a variable number of song sub-folders, and the exact counts per genre were printed during the build_song_index step. The 85/15 train-validation split was applied per genre to maintain class balance. All audio was resampled to 16,000 Hz (mono) before any processing. The 20-second fixed duration was chosen to capture sufficient timbral and rhythmic information — shorter durations like 5 or 10 seconds were briefly experimented with but didn't give stable spectrograms for the transformer model.

3.3 Data Preprocessing & Augmentation

A core part of the pipeline was synthetic mashup generation. Rather than using individual stems directly, during training a "synthetic mashup" was created on-the-fly by loading all four stems for a song, applying per-stem random gain (0.6–1.4), and summing them. This simulates a real mixed track and forces the model to learn from full song representations. Additionally, with a probability of 0.8, a random ESC-50 noise clip was mixed in at low amplitude to improve noise robustness.

SpecAugment was applied on the GPU after mel spectrogram computation — time masking and frequency masking were applied with configurable widths (TIME_MASK and FREQ_MASK parameters in the CNN notebook). The entire mel spectrogram computation pipeline was implemented as a PyTorch nn.Module to run on the GPU, which significantly reduced CPU bottlenecks during training.

4. Feature Extraction Strategy

4.1 Mel Spectrogram (CNN / CNN+GRU Models)

For the custom CNN-based models, log-mel spectrograms were computed using torchaudio on the GPU. The key parameters were: sample rate of 16,000 Hz, 128 mel bins, FFT window size of 1024 samples (64 ms), and a hop length of 512 samples (32 ms). With a 20-second clip this produces approximately 625 time frames. The output is a (1, 128, ~625) tensor which is suitable for 2D convolution. The spectrogram values were log-compressed and passed through a per-batch normalization.

4.2 AST-Compatible Spectrogram

The Audio Spectrogram Transformer has specific preprocessing requirements derived from the ASTFeatureExtractor in HuggingFace. For the AST model, an n_fft of 400 and a hop length of 313 were used, producing 1024 time frames for a 20-second clip. Crucially, the spectrogram was normalized using AST's own constants: mean = -4.2677393 and std = 4.5689974 (using the formula: normalized = (dB_spec - AST_MEAN) / (AST_STD * 2)). This matched the normalization used during the model's original AudioSet pretraining, which was important to ensure the pretrained weights remain meaningful.

4.3 Justification

Log-mel spectrograms are widely considered the most effective representation for audio classification tasks as they roughly approximate the human auditory system. The choice of 128 mel bins captures sufficient frequency resolution for genre discrimination (e.g., distinguishing classical orchestral harmonics from metal distortion). Using the GPU-based transform pipeline (as a custom nn.Module) rather than CPU-based feature extraction was a deliberate engineering choice to avoid DataLoader bottlenecks — all raw waveforms are loaded to RAM first, and spectrograms are computed in batches during the forward pass.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Audio Spectrogram Transformer (AST)

Architecture: The AST model (MIT/ast-finetuned-audioset-10-10-0.4593 from HuggingFace) is a vision transformer adapted for audio. It treats the log-mel spectrogram as an image and applies patch-based self-attention. The pretrained checkpoint was originally trained on AudioSet with 527 output classes. The classification head was replaced with a 10-class head (ignore_mismatched_sizes=True) for the genre task. The model was loaded with DataParallel support for multi-GPU training.

Fine-Tuning Strategy: A two-phase fine-tuning strategy was used. In Phase 1 (5 epochs), the base transformer weights were frozen and only the classification head was trained using AdamW at LR = 5e-4. In Phase 2 (8 epochs), all layers were unfrozen with layer-wise learning rates — the base model used LR = 1e-5 while the classifier head used LR = 1e-4 (10x multiplier). A cosine schedule with 10% warmup steps was applied in both phases. Gradient accumulation (GRAD_ACCUM=2) gave an effective batch size of 32. Mixed precision (autocast + GradScaler) was used throughout.

This two-phase approach is standard practice for fine-tuning large pretrained models on smaller datasets — warming up just the head first prevents catastrophic forgetting of the pretrained representations before the full network adapts together.

5.2 Model 2: Custom Residual CNN (ImprovedCNN)

Architecture: The ImprovedCNN is a fully custom architecture designed to process (1, 128, T_FRAMES) mel spectrograms. It starts with a stem convolution layer followed by three stride-2 residual blocks that progressively halve the spatial dimensions while increasing channel depth. After 4 halvings (stem + 3 blocks), the feature map is spatially reduced to roughly 8 x T/16 before global average pooling and a linear classification head with dropout.

Each residual block follows the standard pre-activation ResNet design: BatchNorm → ReLU → Conv2d → BatchNorm → ReLU → Conv2d, with a skip connection using a 1x1 convolution when the channel dimension changes. The channel progression was 32 → 64 → 128 → 256. This architecture was trained end-to-end from scratch (no pretraining) using AdamW with LR = 1e-3 and CosineAnnealingLR decay down to 1e-6 over the full training duration.

5.3 Model 3: CNN + Bidirectional GRU (ImprovedCNNGRU)

Architecture: The CNN+GRU hybrid model shares the same convolutional front-end as ImprovedCNN but adds a bidirectional GRU layer on top of the CNN feature maps to model temporal dependencies. After the convolutional blocks reduce the spectrogram to (B, C, H, T), the spatial and channel dimensions are merged

and the sequence is passed through a 2-layer Bidirectional GRU. The final hidden states from both directions are concatenated and fed into the classification head.

The rationale is that music genre has both local spectral patterns (handled well by CNN) and longer-range temporal patterns like rhythmic regularity and harmonic progressions (better captured by recurrent modeling). The GRU was chosen over LSTM for its simpler gating (fewer parameters) while still capturing long-range dependencies reasonably well. Training configuration was identical to the CNN model — same optimizer, scheduler, and data pipeline.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary evaluation metric, consistent with the Kaggle competition, was Macro F1 Score — an unweighted average of per-class F1 scores. This is appropriate for multi-class classification where class balance may not be perfect. Additional metrics tracked during training included cross-entropy loss (with label smoothing = 0.1) and per-epoch validation accuracy.

6.2 Training Details Summary

Parameter	AST	CNN	CNN+GRU
Epochs	5 (P1) + 8 (P2)	Variable (early stop)	Variable (early stop)
Optimizer	AdamW	AdamW	AdamW
Learning Rate	5e-4 / 1e-5	1e-3	1e-3
LR Scheduler	Cosine + Warmup	CosineAnnealingLR	CosineAnnealingLR
Batch Size (eff.)	32 (accum x2)	32	32
Label Smoothing	0.1	0.1	0.1
Mixed Precision	Yes (AMP)	Yes (AMP)	Yes (AMP)
TTA at Inference	Yes (x7 crops)	Yes (x7 crops)	Yes (x7 crops)

6.3 Model Comparison

The table below summarizes estimated validation performance observed across models. Exact numbers from WandB logs are referenced in the experiments. Note that the AST model has significantly more parameters (~86M) compared to the custom models (~1-3M), and accordingly takes longer to train.

Model	Val F1 (Macro)	Parameters	Training Time	Complexity
AST (Fine-tuned)	Best (~0.82+)	~86M	~15 min (2xT4)	High
CNN+GRU	Competitive	~3M	~5 min (2xT4)	Medium
Residual CNN	Baseline	~2M	~5 min (2xT4)	Medium

The AST model, as expected, achieves the best performance due to its pretraining on AudioSet which already contains a diverse range of audio including music. The two-phase fine-tuning strategy was key — attempting to fine-tune all layers from the start tended to destabilize training early on.

The CNN+GRU model was the most interesting result. Despite having around 50x fewer parameters than the AST, its validation F1 was considerably competitive. The temporal modeling via the bidirectional GRU appears to add meaningful signal over the pure CNN — especially for genres with distinctive rhythmic patterns like disco, hiphop, and reggae. The pure CNN struggled more with genres that rely on longer-range harmonic or structural patterns.

TTA (Test Time Augmentation) with 7 random crops contributed meaningfully to final test set performance for all three models, as different crop positions reveal different musical segments, and averaging softmax probabilities across crops smooths out prediction noise.

The final submission was generated using the best Phase 2 AST checkpoint with TTA x7.

7. Conclusion & Future Work

7.1 Key Learnings

This project was quite insightful on several fronts. The most important takeaway was understanding how to adapt a large pretrained model (AST, originally trained on AudioSet) to a domain-specific task through careful two-phase fine-tuning. Aggressive fine-tuning from the start led to poor generalization, while the phased approach — head-only first, then full-model — produced more stable results.

Another learning was the value of the GPU-based preprocessing pipeline. Moving the mel spectrogram computation onto the GPU as part of the forward pass (instead of CPU-preprocessing in the DataLoader) essentially eliminated the data pipeline as a bottleneck and significantly improved GPU utilization.

The synthetic mashup generation strategy — combining stems with random gain variations — turned out to be more effective than I initially expected. It forced the model to be robust to different instrument balances and essentially quadrupled the effective training diversity from a fixed set of songs.

7.2 Challenges Encountered

One of the trickier aspects was getting the AST model's input normalization right. Using the wrong normalization constants caused the model to underperform significantly in early experiments, and it took some digging into the HuggingFace ASTFeatureExtractor source code to figure out the exact values (AST_MEAN = -4.2677393, AST_STD = 4.5689974).

Memory management on dual T4 GPUs was also non-trivial. Because the AST model is large and the batch contains audio tensors, OOM errors were common at the start. Gradient accumulation, mixed precision, and pin_memory DataLoader settings together resolved this.

7.3 Areas for Improvement

Several directions could push the score further:

- Ensemble the three models using soft voting on predicted probabilities. Even a simple average of AST and CNN+GRU logits might improve over either individual model.
- Try larger AST variants or other audio transformers like BEATs or HTS-AT, which have been shown to perform better on music tasks.

- More aggressive hyperparameter search — particularly for the CNN+GRU model, which was not tuned as carefully as the AST.
- Experiment with longer audio durations (e.g., 30 seconds) or multi-crop training, which may capture more musical structure per sample.
- Explore Mixup augmentation at the waveform level, which has shown strong results in audio classification literature.

8. References

[1] Gong, Y., Liu, S., & Glass, J. (2021). AST: Audio Spectrogram Transformer. arXiv:2104.01778.

[2] Park, D. S., et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech 2019.

[3] HuggingFace Transformers library. https://huggingface.co/MIT/ast-finetuned-audioset-10-10-0.4593

[4] PyTorch torchaudio documentation. https://pytorch.org/audio/